

BỘ GIÁO DỤC VÀ ĐÀO TẠO — TRƯỜNG ĐẠI HỌC

ĐỘC LẬP – TỰ DO – HẠNH PHÚC



SỔ TAY KỸ THUẬT & VẬN HÀNH MÁY CHỦ

CÀI ĐẶT MÔI TRƯỜNG, DOCKER & ỨNG CỨU SỰ CỐ

ĐỐI TƯỢNG VÀ PHẠM VI ÁP DỤNG:

- Đội ngũ Kỹ sư Trung tâm CNTT, Quản trị viên Mạng & DevOps
- Chuẩn hóa theo Quy chế Khảo thí Đại học hiện hành
- Hỗ trợ khảo thí trực tuyến, chống gian lận & Trợ lý AI

HỆ THỐNG QUẢN LÝ KHẢO THÍ SINH VIÊN (EXAM MANAGEMENT SYSTEM)

Phiên bản Phát hành: Version 1.0 — Năm học 2025 – 2026

Lưu hành nội bộ — Xuất bản ngày 3/9/2026

MỤC LỤC TỔNG THỂ

Phần 4: Sổ Tay Kỹ Thuật & Vận Hành Khẩn Cấp (IT / DevOps)

- 01. SỔ TAY CÀI ĐẶT & CẤU HÌNH MÔI TRƯỜNG VẬN HÀNH (DEV / PRODUCTION)
- 02. HƯỚNG DẪN TRIỂN KHAI & VẬN HÀNH HỆ THỐNG BẰNG DOCKER
- 03. SỔ TAY XỬ LÝ SỰ CỐ KHẨN CẤP DÀNH CHO KỸ SƯ HỆ THỐNG (IT RUNBOOK)

01. SỔ TAY CÀI ĐẶT & CẤU HÌNH MÔI TRƯỜNG VẬN HÀNH (DEV / PRODUCTION)

Tài liệu này dành riêng cho Kỹ sư IT, Quản trị viên hệ thống (DevOps / SysAdmin) để cài đặt, cấu hình và khởi chạy hệ thống khảo thí trên máy chủ Windows hoặc Linux (Ubuntu/Debian/RHEL).



1. Yêu Cầu Phần Cứng & Phần Mềm Nền Tảng

Yêu cầu phần cứng khuyến nghị:

Môi trường Phát triển (Dev / Testing): CPU 4 Cores, RAM tối thiểu 8GB, Ổ cứng SSD còn trống tối thiểu 20GB.

Môi trường Vận hành Chính thức (Production - Cho 1.000+ thí sinh đồng thời):

CPU: 8 đến 16 Cores (Intel Xeon / AMD EPYC).

RAM: 16GB đến 32GB RAM.

Ổ cứng: NVMe SSD 100GB+ (Tốc độ đọc/ghi cao phục vụ ghi Audit Log và Database I/O).

Băng thông mạng: Tối thiểu 1 Gbps.

Yêu cầu phần mềm bắt buộc:

Node.js: Phiên bản LTS ổn định **v18.x** hoặc **v20.x** (Khuyến nghị Node.js 20.12+ LTS).

NPM: Phiên bản 9.x hoặc 10.x (kèm theo Node.js).

Cơ sở dữ liệu: PostgreSQL phiên bản 15 hoặc 16.

Git: Phiên bản mới nhất.



2. Cấu Hình Biến Môi Trường (.env)

Hệ thống tách biệt rõ ràng biến môi trường giữa Backend và Frontend:

A. Cấu hình Backend (backend/ .env):

Sao chép từ file mẫu backend/.env.example :

```
cd backend
cp .env.example .env
```

Các tham số quan trọng cần thiết lập trong `backend/.env` :

```
# Cổng dịch vụ API Backend
PORT=3001

# Chuỗi kết nối Cơ sở Dữ liệu PostgreSQL
DATABASE_URL="postgresql://postgres:MatKhouManh123@localhost:5432/exam_management_db?schema=publ

# Khóa bí mật mã hóa JWT (Bắt buộc dùng chuỗi ngẫu nhiên tối thiểu 32 ký tự)
JWT_SECRET="super-secret-jwt-key-for-exam-production-2026"
JWT_EXPIRES_IN="7d"

# Cấu hình AI Provider chấm thi tự luận (Gemini hoặc DeepSeek)
AI_PROVIDER="gemini"
GEMINI_API_KEY="AIzaSyYourGeminiApiKeyHere..."

# Cấu hình thư mục lưu trữ bản sao lưu CSDL
BACKUP_STORAGE_DIR="./database-backups"
```

B. Cấu hình Frontend (`frontend/.env.local`):

```
# Địa chỉ API Gateway backend mà trình duyệt gọi tới
NEXT_PUBLIC_API_URL="http://localhost:3001"
```

3. Hướng Dẫn Cài Đặt & Khởi Chạy Từng Bước

Tại thư mục gốc của dự án (`exam-management/`):

Bước 1: Cài đặt toàn bộ thư viện phụ thuộc (Dependencies)

```
npm run install:all
```

Lệnh này sẽ tự động cài đặt `node_modules` cho cả thư mục `backend` và `frontend` .

Bước 2: Khởi tạo Cơ sở dữ liệu & Seed dữ liệu mẫu ban đầu

```
# Tạo cấu trúc bảng từ schema.prisma vào PostgreSQL
npm run db:migrate

# Bơm dữ liệu mẫu (Khoa, Lớp, Môn học, Tài khoản Admin, GV, SV)
npm run seed
```

Bước 3: Khởi chạy Môi trường Phát triển (Development Mode)

```
npm run dev
```

Lệnh này dùng `concurrently` để chạy song song:

Backend API: `http://localhost:3001`

Frontend Web UI: `http://localhost:3000`



4. Đóng Gói & Chạy Môi Trường Vận Hành (Production Mode)

Khi đưa lên máy chủ chính thức của trường:

```
# 1. Sinh Prisma Client và Biên dịch toàn bộ mã nguồn
npm run build:all

# 2. Khởi chạy ở chế độ Production tối ưu hiệu năng
npm run start
```

💡 Mẹo hay

Trên môi trường Production Linux, khuyến nghị sử dụng công cụ quản lý tiến trình **PM2** (`pm2 start npm --name "exam-backend" -- run start:prod --prefix backend`) để tự động khởi động lại ứng dụng nếu gặp sự cố sập nguồn hoặc crash bộ nhớ.

02. HƯỚNG DẪN TRIỂN KHAI & VẬN HÀNH HỆ THỐNG BẰNG DOCKER

Tài liệu này hướng dẫn Kỹ sư DevOps triển khai Hệ thống Khảo thí bằng công nghệ container hóa **Docker** và **Docker Compose**, giúp hệ thống vận hành cô lập, ổn định và dễ dàng mở rộng.

1. Mô Hình Các Dịch Vụ Container (Architecture)

Hệ thống được đóng gói thành cụm 3 container liên kết nội bộ trong mạng riêng `exam-network` :

```
graph TD
    User["🌐 Người Dùng / Trình Duyệt Thí Sinh & Cán Bộ"] -->|Port 80/443| Nginx["🔵 Nginx Reverse Proxy"]
    Nginx -->|Port 3000| Frontend["💻 Container: exam-frontend-web (Next.js 14 Standalone)"]
    Nginx -->|Port 3001| Backend["⚡ Container: exam-backend-api (NestJS API Gateway)"]
    Backend -->|Port 5432| DB["🐘 Container: exam-postgres-db (PostgreSQL 16 Alpine)"]

    DB --> VolDB["💾 Volume: postgres_data"]
    Backend --> VolUploads["📁 Volume: backend_uploads"]
    Backend --> VolBackups["📦 Volume: backend_backups"]
```

2. Các Lệnh Điều Hành Docker Nhanh

Tại thư mục gốc dự án (`exam-management/`), bạn có thể sử dụng các lệnh script có sẵn trong `package.json` :

Thao tác	Lệnh NPM viết tắt	Lệnh Docker Compose gốc
Khởi chạy ngầm toàn bộ	<code>npm run docker:up</code>	<code>docker compose up -d</code>
Dừng & Hủy container	<code>npm run docker:down</code>	<code>docker compose down</code>
Đóng gói lại Image (Build)	<code>npm run docker:build</code>	<code>docker compose build</code>
Xem Log trực tiếp (Live Logs)	-	<code>docker compose logs -f</code>
Xem Log riêng Backend	-	<code>docker compose logs -f backend</code>
Xem Trạng thái các container	-	<code>docker compose ps</code>

3. Quản Lý Dữ Liệu Tồn Tại Lâu Dài (Persistent Volumes)

Để dữ liệu không bao giờ bị mất khi khởi động lại hoặc cập nhật phiên bản container mới, 3 volume chuyên dụng được liên kết với máy chủ chủ (Host Machine):

postgres_data : Lưu trữ toàn bộ bảng biểu, tài khoản, điểm số của PostgreSQL tại `/var/lib/postgresql/data` . Tuyệt đối không xóa volume này trừ khi muốn làm mới hoàn toàn hệ thống.

backend_uploads : Lưu trữ các file đính kèm, ảnh chụp minh chứng đề thi, bài làm tự luận của sinh viên.

backend_backups : Nơi lưu các file backup `.sql` tự động do worker sinh ra. Kỹ sư IT có thể copy dữ liệu từ thư mục này sang ổ đĩa ngoài định kỳ.

4. Kiểm Tra Sức Khỏe & Truy Cập Container (Healthcheck & Exec)

Kiểm tra cơ sở dữ liệu PostgreSQL đã sẵn sàng chưa:

Container `exam-postgres-db` có tích hợp cơ chế Healthcheck tự động:

```
docker exec -it exam-postgres-db pg_isready -U postgres -d exam
```

Kết quả trả về: `exam:5432 - accepting connections` nghĩa là CSDL đang hoạt động hoàn hảo.

Mở dòng lệnh Terminal bên trong Container Backend:

Khi cần chạy các lệnh đặc biệt như seed dữ liệu hoặc kiểm tra log file:

```
docker exec -it exam-backend-api sh

# Chạy seed dữ liệu thủ công bên trong container:
npm run seed
```

5. Cấu Hình Nginx Reverse Proxy & Chứng Chỉ SSL HTTPS

Khi triển khai trên máy chủ thật có tên miền chính thức (Ví dụ: `khaothi.truongdaihoc.edu.vn`), khuyến nghị thiết lập cấu hình Nginx phía trước:

```

server {
    listen 80;
    server_name khaothi.truongdaihoc.edu.vn;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name khaothi.truongdaihoc.edu.vn;

    ssl_certificate /etc/letsencrypt/live/khaothi.truongdaihoc.edu.vn/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/khaothi.truongdaihoc.edu.vn/privkey.pem;

    # Chuyển tiếp các request thông thường về Frontend Next.js
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }

    # Chuyển tiếp các request API hoặc WebSocket về Backend NestJS
    location /api/ {
        proxy_pass http://127.0.0.1:3001/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 300s;
    }
}

```

Mẹo hay

Sử dụng công cụ miễn phí **Certbot** (`certbot --nginx -d khaothi.truongdaihoc.edu.vn`) để tự động cài đặt và gia hạn chứng chỉ bảo mật SSL Let's Encrypt 90 ngày một lần.

03. SỔ TAY XỬ LÝ SỰ CỐ KHẨN CẤP DÀNH CHO KỸ SƯ HỆ THỐNG (IT RUNBOOK)

Tài liệu này là cẩm nang phản ứng nhanh (**Incident Response Runbook**) dành cho đội ngũ Quản trị IT / DevOps khi hệ thống gặp trục trặc trong quá trình tổ chức kỳ thi.

1. Sự Cố 1: Backend API Bị Crash Hoặc Thoát Đột Ngột

Hiện tượng:

Frontend báo lỗi *"Network Error"* hoặc *"502 Bad Gateway"*. Terminal hoặc Docker báo backend dừng với exit code 1.

Quy trình chẩn đoán & Khắc phục:

Kiểm tra nhật ký lỗi tức thời:

```
# Nếu chạy Docker:
docker compose logs --tail=100 backend

# Nếu chạy PM2:
pm2 logs exam-backend
```

Các nguyên nhân phổ biến:

Tràn bộ nhớ RAM (Out of Memory - OOM): Do hàng trăm sinh viên cùng nộp bài tự luận chứa ảnh dung lượng lớn trong cùng một giây.

Giải pháp: Tăng giới hạn RAM cho tiến trình Node.js bằng cờ:

```
NODE_OPTIONS="--max-old-space-size=4096" npm run start:prod
```

Thiếu biến môi trường hoặc sai cấu hình JWT: Kiểm tra xem file `backend/.env` có bị xóa nhầm hoặc thiếu `JWT_SECRET` hay không.

Khởi động lại dịch vụ ngay lập tức:

```
docker compose restart backend
```

2. Sự Cố 2: Lỗi Kết Nối Cơ Sở Dữ Liệu PostgreSQL (Prisma P1001)

Hiện tượng:

Log backend xuất hiện lỗi: `PrismaClientInitializationError: Can't reach database server at 'db:5432'.`

Quy trình chẩn đoán & Khắc phục:

Kiểm tra tiến trình PostgreSQL:

```
docker ps | grep postgres
```

Nếu container bị tắt, kiểm tra nguyên nhân sập máy:

```
docker compose logs db
```

Tràn số lượng kết nối tối đa (`Too many connections`):

Mặc định PostgreSQL chỉ cho phép 100 kết nối đồng thời. Trong kỳ thi lớn, hàng chục tiến trình backend có thể làm cạn kiệt Connection Pool.

Khắc phục: Tăng `max_connections` trong file cấu hình PostgreSQL hoặc khởi chạy lại container với tham số:

```
command: postgres -c max_connections=300 -c shared_buffers=512MB
```

Điều chỉnh chuỗi kết nối Prisma trong `.env` :

```
DATABASE_URL="postgresql://user:pass@db:5432/exam?
schema=public&connection_limit=50"
```

3. Sự Cố 3: Quá Tải Đột Biến Đầu Giờ Thi (Traffic Spike)

Hiện tượng:

Vào đúng thời điểm 07:30 (khi giám thị đọc mật khẩu đề thi), 1.000 đến 2.000 sinh viên cùng nhấn F5 hoặc bấm nút "Bắt đầu làm bài", khiến CPU máy chủ chạm ngưỡng 100%, trang web phản hồi chậm.

Các biện pháp giải tỏa tắc nghẽn khẩn cấp:

Bật bộ đệm tĩnh (Static Caching) trên Nginx:

Đảm bảo toàn bộ tài nguyên CSS, JS, hình ảnh được Nginx lưu cache trực tiếp trên RAM, không đẩy request về Node.js.

Kỹ thuật Mở ca thi So le (Staggered Schedule):

Phòng Khảo thí nên cấu hình thời gian bắt đầu làm bài của các Khoa lệch nhau từ 5 đến 10 phút (Ví dụ: Khoa CNTT thi lúc 07:30, Khoa Kinh tế thi lúc 07:40). Biện pháp này triệt tiêu hoàn toàn đỉnh nhọn quá tải (Traffic Spike) mà không cần tốn thêm chi phí nâng cấp phần cứng máy chủ.

4. Sự Cố 4: Khôi Phục Dữ Liệu Khẩn Cấp Bằng Lệnh Dòng Lệnh (CLI Restore)

Nếu giao diện web bị lỗi hoàn toàn và không thể truy cập vào trang `/admin/backups`, Kỹ sư IT có thể khôi phục database trực tiếp bằng công cụ `psql`:

```
# Bước 1: Xác định file sao lưu gần nhất trong thư mục database-backups/  
ls -lh database-backups/  
  
# Bước 2: Bơm dữ liệu từ file backup vào PostgreSQL (Áp dụng cho Docker):  
docker exec -i exam-postgres-db psql -U postgres -d exam < database-backups/backup-2026-09-02-02  
  
# Bước 3: Khởi động lại backend để nhận dữ liệu mới:  
docker compose restart backend
```

5. Bảng Kiểm Tra Sẵn Sàng Trước Giờ Thi (Pre-Flight Checklist)

Đội ngũ kỹ thuật IT phải hoàn thành kiểm tra 6 mục này tối thiểu **30 phút trước mỗi ca thi**:

- [] **1. Dung lượng ổ cứng:** Kiểm tra ổ cứng máy chủ còn trống tối thiểu **20GB** (`df -h`).
- [] **2. Sao lưu CSDL:** Đã có bản backup sạch sẽ của CSDL được tạo trong vòng 24 giờ qua.
- [] **3. Kết nối mạng phòng máy:** Đảm bảo toàn bộ switch mạng, dây cáp mạng và máy trạm tại các phòng LAB thông suốt tới máy chủ.
- [] **4. Kiểm tra tài khoản Quản trị:** Đăng nhập thử vào trang Admin, kiểm tra đồng hồ máy chủ đã được đồng bộ chuẩn giờ internet qua NTP.
- [] **5. Mở cổng Firewall:** Đảm bảo các cổng `80` , `443` , `3000` , `3001` không bị tường lửa Windows Defender hoặc iptables chặn ngoài ý muốn.
- [] **6. Cán bộ thường trực:** Tối thiểu 2 kỹ sư kỹ thuật túc trực tại phòng điều khiển máy chủ trong suốt thời gian diễn ra ca thi để kịp thời ứng cứu khi có sự cố.